

LAPORAN PRAKTIKUM

TPL1109 Algoritma dan Struktur Data

Pertemuan 11 - Representasi Struktur Data Graph Menggunakan Python

Nama : Mohammad Azmi Zaeni
NIM : J0403251120
Kelas : TPL A1
Dosen : Dr. Shelve Nidya Neyman, S.Kom, M.Si

Praktikum 4 - Studi Kasus Dunia Nyata

Langkah 1 - Studi Kasus

Digit akhir NIM: 0 → Studi Kasus: Media Sosial

Studi kasus yang dipilih adalah representasi jaringan media sosial berbasis follow, mirip dengan platform seperti Twitter atau Instagram. Hubungan antar pengguna bersifat satu arah (directed), artinya jika pengguna A mengikuti pengguna B, belum tentu B mengikuti A.

Langkah 2 - Node dan Edge

A. Node (Vertex) - Pengguna Media Sosial:

- Yusuf
- Adit
- Hanif
- Haidar
- Pakdeh

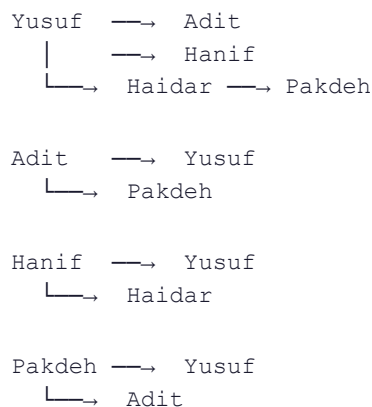
B. Edge - Hubungan Follow (Satu Arah):

No	Dari (User)	Ke (User)	Keterangan
1	Yusuf	Adit	Yusuf follow Adit
2	Yusuf	Hanif	Yusuf follow Hanif
3	Yusuf	Haidar	Yusuf follow Haidar
4	Adit	Yusuf	Adit follow Yusuf
5	Adit	Pakdeh	Adit follow Pakdeh
6	Hanif	Yusuf	Hanif follow Yusuf
7	Hanif	Haidar	Hanif follow Haidar
8	Haidar	Pakdeh	Haidar follow Pakdeh

9	Pakdeh	Yusuf	Pakdeh follow Yusuf
10	Pakdeh	Adit	Pakdeh follow Adit

Langkah 3 - Desain Graph

Berikut adalah representasi sederhana graph media sosial ini (directed graph):



Langkah 4 - Implementasi Python

A. Adjacency List:

Node (User)	Mengikuti (Following)
Yusuf	Adit, Hanif, Haidar
Adit	Yusuf, Pakdeh
Hanif	Yusuf, Haidar
Haidar	Pakdeh
Pakdeh	Yusuf, Adit

Implementasi dalam Python:

```

#Nama: Mohammad Azmi Zaeni
#NIM: J0403251120
#Kelas: TPL A1
#=====

# Praktikum 4 - Studi Kasus Dunia Nyata
# Studi Kasus: Media Sosial (Directed Graph)

def createAdjList(edges):
    adj = {
        "Yusuf": [],
        "Adit": [],
        "Hanif": [],

```

```

        "Haidar": [],
        "Pakdeh": []
    }
    for u, v in edges:
        adj[u].append(v)
    return adj

def createAdjMatrix(nodes, edges):
    n = len(nodes)
    matrix = [[0] * n for _ in range(n)]
    index = {node: i for i, node in enumerate(nodes)}
    for u, v in edges:
        matrix[index[u]][index[v]] = 1
    return matrix

if __name__ == "__main__":
    nodes = ["Yusuf", "Adit", "Hanif", "Haidar", "Pakdeh"]
    edges = [
        ("Yusuf", "Adit"), ("Yusuf", "Hanif"), ("Yusuf", "Haidar"),
        ("Adit", "Yusuf"), ("Adit", "Pakdeh"),
        ("Hanif", "Yusuf"), ("Hanif", "Haidar"),
        ("Haidar", "Pakdeh"),
        ("Pakdeh", "Yusuf"), ("Pakdeh", "Adit"),
    ]
    adj = createAdjList(edges)
    print('Adjacency List Representation:')
    for user, following in adj.items():
        print(f'{user} -> {following}')

    matrix = createAdjMatrix(nodes, edges)
    print('\nAdjacency Matrix Representation:')
    print(f"{'':>8}", end='')
    for node in nodes:
        print(f'{node:>8}', end='')
    print()
    for i, node in enumerate(nodes):
        print(f'{node:>8}', end='')
        for j in range(len(nodes)):
            print(f'{matrix[i][j]:>8}', end='')
        print()

```

B. Adjacency Matrix:

	Yusuf	Adit	Hanif	Haidar	Pakdeh
Yusuf	0	1	1	1	0
Adit	1	0	0	0	1
Hanif	1	0	0	1	0
Haidar	0	0	0	0	1
Pakdeh	1	1	0	0	0

Keterangan: nilai 1 (biru) = ada hubungan follow, nilai 0 (abu) = tidak ada hubungan.

Langkah 5 - Output Program

Adjacency List Output:

```
Adjacency List Representation:
Yusuf  -> ['Adit', 'Hanif', 'Haidar']
Adit   -> ['Yusuf', 'Pakdeh']
Hanif  -> ['Yusuf', 'Haidar']
Haidar -> ['Pakdeh']
Pakdeh -> ['Yusuf', 'Adit']
```

Adjacency Matrix Output:

	Yusuf	Adit	Hanif	Haidar	Pakdeh
Yusuf	0	1	1	1	0
Adit	1	0	0	0	1
Hanif	1	0	0	1	0
Haidar	0	0	0	0	1
Pakdeh	1	1	0	0	0

Langkah 6 - Analisis Singkat

1. Jenis Graph

Graph yang digunakan pada studi kasus ini adalah Directed Graph (graf berarah). Hubungan follow pada media sosial bersifat satu arah, artinya jika Yusuf mengikuti Adit, belum tentu Adit mengikuti Yusuf kembali. Hal ini berbeda dengan undirected graph yang hubungannya selalu bersifat dua arah (seperti pertemanan di Facebook).

2. Alasan Pemilihan Edge

Edge dipilih berdasarkan logika hubungan follow pada platform media sosial seperti Twitter dan Instagram. Setiap pasangan edge (u, v) merepresentasikan tindakan user u mengikuti user v . Edge tidak perlu simetris karena dalam kehidupan nyata, hubungan follow tidak selalu berbalas. Dengan 5 node dan 10 edge, graph ini memiliki jumlah koneksi yang cukup variatif untuk merepresentasikan aktivitas sosial sederhana.

3. Representasi yang Lebih Cocok

Untuk studi kasus media sosial ini, Adjacency List lebih cocok digunakan dibandingkan Adjacency Matrix. Berikut alasannya:

Aspek	Adjacency List	Adjacency Matrix
Bentuk	Dictionary (key-value)	Matriks 2D (list of list)
Memori	Hemat (hanya simpan edge yang ada)	Boros (semua kemungkinan disimpan)
Cek edge	Lebih lambat $O(V)$	Sangat cepat $O(1)$
Cocok untuk	Sparse graph	Dense graph
Kompleksitas ruang	$O(V+E)$	$O(V^2)$

Pada media sosial nyata, jumlah pengguna bisa mencapai jutaan, namun setiap pengguna hanya mengikuti sebagian kecil pengguna lainnya (sparse graph). Jika menggunakan adjacency matrix dengan 1 juta user, dibutuhkan ruang $1.000.000 \times 1.000.000 = 10^{12}$ sel, yang sangat boros memori. Sebaliknya, adjacency list hanya menyimpan edge yang benar-benar ada.

4. Dense atau Sparse Graph

Graph pada studi kasus ini termasuk Sparse Graph. Dari 5 node yang ada, kemungkinan edge maksimal (directed) adalah $5 \times 4 = 20$ edge. Namun pada implementasi ini hanya terdapat 10 edge, sehingga hanya 50% dari kapasitas maksimal. Pada skala nyata, rasio ini akan jauh lebih kecil lagi karena setiap pengguna hanya mengikuti sebagian kecil dari seluruh pengguna yang ada.

Kesimpulan

Dari praktikum ini dapat disimpulkan beberapa hal penting:

1. Graph adalah struktur data non-linear yang sangat berguna untuk merepresentasikan hubungan antar entitas di dunia nyata, termasuk media sosial.

2. Directed graph digunakan ketika hubungan antar node bersifat satu arah, seperti hubungan follow pada media sosial.
3. Adjacency List lebih efisien dari segi memori untuk sparse graph karena hanya menyimpan edge yang benar-benar ada, dengan kompleksitas ruang $O(V+E)$.
4. Adjacency Matrix lebih cocok untuk dense graph dan ketika pengecekan koneksi antar node perlu dilakukan dengan cepat $O(1)$.
5. Pemahaman representasi graph (adjacency list dan matrix) adalah fondasi penting sebelum mempelajari algoritma traversal seperti BFS dan DFS.